

École Marocaine des Sciences de l'Ingénieur
EMSI Rabat

Rapport de Projet de Fin d'Année

Previzma

Application Web de Prédiction des Ventes pour l'Industrie B2B

Prédire l'avenir, anticiper les décisions.

Réalisé par :

MALKI Achraf

TAIBI ALAOUI Yazid

Encadré par :

Mme. AARICH Mounia

Filière : Cycle Ingénieur — Informatique et Réseaux

Année universitaire : 2025–2026

Remerciements

Nous tenons à exprimer nos sincères remerciements à notre encadrante, Mme. AARICH Mounia, pour son accompagnement, ses orientations et ses remarques constructives tout au long de la réalisation de ce projet de fin d'année. Son suivi nous a permis de structurer progressivement notre réflexion, depuis l'identification de la problématique jusqu'à la conception de la solution.

Nous remercions également l'École Marocaine des Sciences de l'Ingénieur de Rabat pour le cadre pédagogique et technique mis à notre disposition. Les enseignements reçus en développement logiciel, bases de données, architecture des systèmes d'information, intelligence artificielle et génie logiciel ont constitué une base importante pour mener ce projet.

Enfin, nous adressons nos remerciements à toutes les personnes ayant contribué, directement ou indirectement, à la maturation de ce travail, notamment à travers les échanges autour des problématiques liées aux systèmes ERP, à l'industrie B2B et à la transformation des données opérationnelles en informations décisionnelles.

Résumé

La prévision des ventes représente un enjeu stratégique pour les entreprises industrielles opérant en B2B. Dans ce contexte, les cycles de vente sont souvent longs, les volumes importants, les portefeuilles clients hétérogènes et les décisions commerciales fortement dépendantes de la disponibilité des données historiques. Cependant, les données issues des systèmes ERP ne sont pas directement exploitables par les modèles de Machine Learning : elles nécessitent une transformation métier permettant de passer d'une logique transactionnelle à une logique analytique.

Ce rapport présente Previzma, une application web de prévision des ventes pour l'industrie B2B. Le projet propose une architecture distribuée composée d'un frontend Angular, d'un backend Spring Boot, d'une base de données PostgreSQL/Supabase et d'un microservice de prévision basé sur FastAPI et XGBoost. La solution intègre une segmentation client B2B, un dashboard décisionnel multi-KPI, un système d'alertes opérationnelles et un module *What-If* permettant de simuler l'impact de décisions commerciales sur les ventes prévues.

L'objectif de Previzma est de fournir aux managers commerciaux un outil lisible, explicable et orienté décision, capable de transformer des données de ventes historiques en prévisions exploitables. Le projet s'appuie sur un dataset de ventes enrichi par une couche métier B2B afin de reproduire un contexte industriel marocain.

Mots-clés : Prévision des ventes, B2B, ERP, Machine Learning, XGBoost, Angular, Spring Boot, FastAPI, PostgreSQL, Dashboard décisionnel.

Abstract

Sales forecasting is a strategic challenge for industrial companies operating in a B2B environment. In this context, sales cycles are often long, volumes are significant, customer portfolios are heterogeneous, and business decisions strongly depend on the availability and quality of historical data. However, ERP data is not directly usable by Machine Learning models. It must first be transformed from an operational and transactional representation into analytical features that can support prediction.

This report presents Previzma, a web application for B2B industrial sales forecasting. The project relies on a distributed architecture composed of an Angular frontend, a Spring Boot backend, a PostgreSQL/Supabase database, and a FastAPI microservice using XGBoost for prediction and simulation. The solution includes native B2B customer segmentation, a multi-KPI decision dashboard, operational alerts, and a What-If module for simulating the impact of business decisions on forecasted sales.

The goal of Previzma is to provide sales managers with a readable, explainable and decision-oriented tool capable of transforming historical sales data into actionable forecasts. The project is based on a sales dataset enriched with B2B business logic in order to reproduce a Moroccan industrial context.

Keywords : Sales forecasting, B2B, ERP, Machine Learning, XGBoost, Angular, Spring Boot, FastAPI, PostgreSQL, Decision Dashboard.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
1 Introduction générale	1
1.1 Contexte général	1
1.2 Présentation du projet	2
1.3 Motivation du projet	2
1.4 Organisation du rapport	2
2 Problématique et objectifs	4
2.1 Problématique métier	4
2.2 Limites des approches existantes	4
2.3 Objectif général	5
2.4 Objectifs spécifiques	5
2.5 Contributions attendues	5
3 Étude théorique et état de l’art	6
3.1 Prévision des ventes	6
3.2 Spécificités du B2B industriel	6
3.3 Données ERP et données Machine Learning	7
3.4 Algorithmes de prévision	7
3.4.1 ARIMA	7
3.4.2 Lissage exponentiel	7
3.4.3 Random Forest	7
3.4.4 XGBoost	8
3.4.5 LSTM	8
3.5 Comparaison synthétique des algorithmes	8
3.6 Solutions existantes	9
3.7 Positionnement de Previzma	9

4	Technologies utilisées	10
4.1	Vue d'ensemble de la stack technique	10
4.2	Angular 18	10
4.3	Spring Boot 3.x	11
4.4	PostgreSQL et Supabase	11
4.5	FastAPI	11
4.6	XGBoost	11
4.7	Outils complémentaires	12
5	Analyse et conception	13
5.1	Besoins fonctionnels	13
5.2	Besoins non fonctionnels	14
5.3	Acteurs du système	14
5.4	Diagrammes de cas d'utilisation	15
5.5	Diagramme de classes	16
5.6	Modèle conceptuel de données	17
5.7	Diagramme d'architecture globale	19
5.8	Diagramme de séquence – Simulation What-If	20
5.9	Pipeline de prévision	20
6	Réalisation de l'application	22
6.1	Préparation des données	22
6.2	Backend Spring Boot	22
6.3	Microservice ML FastAPI	23
6.4	Module What-If	23
6.5	Frontend Angular	24
6.6	Dashboard décisionnel	24
6.7	Sécurité et authentification	24
7	Étude de cas et résultats	26
7.1	Présentation du dataset	26
7.2	Scénario métier étudié	26
7.3	Indicateurs d'évaluation	26
7.4	Résultats de prévision	27
7.5	Simulation What-If	28
7.6	Interprétation des résultats	28
8	Tests, limites et perspectives	29
8.1	Tests réalisés	29
8.2	Limites du projet	30

8.3 Perspectives d'amélioration	30
Conclusion générale	31
Bibliographie	32
A Annexes	33
A.1 Exemples d'endpoints API	33
A.2 Emplacements prévus pour les captures d'écran	33
A.3 Noms recommandés pour les figures externes	33

Table des figures

3.1	Passage des données ERP aux données exploitables par le Machine Learning	7
4.1	Vue d'ensemble de la stack technique de Previzma	10
5.1	Diagramme de cas d'utilisation – Administration et données	15
5.2	Diagramme de cas d'utilisation – Pilotage, prévision et simulation	16
5.3	Diagramme de classes métier de Previzma	17
5.4	Modèle conceptuel de données de Previzma	18
5.5	Diagramme d'architecture globale de Previzma	20
5.6	Diagramme de séquence du module What-If	20
5.7	Pipeline de prévision de Previzma	21
7.1	Comparaison indicative des MAPE par algorithme	27

Liste des tableaux

3.1	Comparaison synthétique des algorithmes de prévision	8
5.1	Besoins fonctionnels de Previzma	13
5.2	Besoins non fonctionnels de Previzma	14
5.3	Entités principales de la base de données	19
7.1	Indicateurs d'évaluation du modèle	27
8.1	Plan de tests prévu	29
A.1	Endpoints principaux de l'application	33
A.2	Fichiers image attendus	34

Chapitre 1

Introduction générale

1.1 Contexte général

La transformation numérique des entreprises industrielles a profondément modifié la manière dont les organisations collectent, traitent et exploitent leurs données. Les systèmes d'information modernes, notamment les ERP, CRM et plateformes de gestion commerciale, centralisent un volume important de données liées aux clients, aux produits, aux commandes, aux ventes et aux opérations logistiques.

Dans l'industrie B2B, ces données constituent un actif stratégique. Elles permettent d'analyser les performances commerciales passées, de comprendre les comportements d'achat, d'anticiper la demande et d'améliorer la planification des ressources. Néanmoins, leur exploitation reste souvent limitée à des rapports descriptifs ou à des tableaux de bord statiques. Les décisions restent alors largement dépendantes de l'expérience des managers, de l'intuition commerciale ou d'extrapolations simples.

La prévision des ventes apporte une réponse à cette limite. Elle vise à estimer les ventes futures à partir des historiques disponibles et de variables explicatives telles que la saisonnalité, le segment client, le prix, la catégorie de produit ou encore les tendances d'achat. Grâce aux avancées du Machine Learning, il devient possible de construire des modèles capables d'identifier des relations non linéaires et des signaux faibles dans les données commerciales.

Cependant, un écart important subsiste entre les données opérationnelles disponibles dans les ERP et les données réellement exploitables par un modèle de prévision. Un ERP stocke principalement des transactions, tandis qu'un modèle de Machine Learning nécessite des variables structurées, nettoyées et porteuses de signaux prédictifs. C'est dans cet espace intermédiaire que s'inscrit le projet Previzma.

1.2 Présentation du projet

Previzma est une application web de prévision des ventes destinée aux entreprises industrielles opérant en B2B. Elle vise à aider les responsables commerciaux et les managers opérationnels à anticiper la demande, suivre les performances par segment client, détecter les risques et simuler l'impact de décisions commerciales.

Le projet se distingue par trois choix structurants. D'abord, il introduit une segmentation B2B native distinguant les grands comptes, les PME et les distributeurs, car ces profils n'ont pas les mêmes rythmes d'achat ni les mêmes contraintes. Ensuite, il propose un dashboard décisionnel multi-KPI conçu pour une lecture rapide par des utilisateurs non techniques. Enfin, il intègre un module *What-If* permettant de simuler l'impact d'un changement de prix, de volume ou de conditions commerciales sur les prévisions.

Sur le plan technique, Previzma repose sur une architecture distribuée combinant Angular pour l'interface utilisateur, Spring Boot pour le backend métier, PostgreSQL/Supabase pour la persistance des données et FastAPI avec XGBoost pour la couche de prévision. Cette séparation permet de faire évoluer indépendamment l'interface, les règles métier, la base de données et le moteur d'intelligence artificielle.

1.3 Motivation du projet

La motivation principale de ce projet provient du constat suivant : les entreprises industrielles disposent souvent de données commerciales riches, mais ces données ne sont pas directement exploitables pour la prévision. Les statuts de commande, les segments clients, les prix, les volumes ou les historiques de vente doivent être transformés pour devenir des variables pertinentes.

Dans un contexte industriel B2B, cette transformation est essentielle. Un grand compte n'a pas le même comportement d'achat qu'une PME ou qu'un distributeur régional. De même, une commande en brouillon, une commande confirmée et une commande annulée ne doivent pas être interprétées de la même manière par le modèle. La valeur du projet réside donc autant dans la couche métier de transformation que dans le modèle de prévision lui-même.

1.4 Organisation du rapport

Ce rapport est structuré en huit chapitres. Le premier chapitre présente le contexte général, le sujet et la motivation du projet. Le deuxième chapitre détaille la problématique et les objectifs. Le troisième chapitre expose l'étude théorique et l'état de l'art. Le quatrième

chapitre décrit les technologies utilisées. Le cinquième chapitre présente l'analyse et la conception. Le sixième chapitre détaille la réalisation de l'application. Le septième chapitre est consacré à l'étude de cas et aux résultats. Enfin, le huitième chapitre présente les tests, les limites et les perspectives d'amélioration.

Chapitre 2

Problématique et objectifs

2.1 Problématique métier

Dans les entreprises industrielles B2B, la prévision des ventes est une activité critique. Elle influence directement la planification de la production, la gestion des stocks, l'allocation des ressources commerciales, la négociation avec les fournisseurs et la satisfaction des clients. Une mauvaise prévision peut entraîner des surstocks, des ruptures, une mauvaise allocation des ressources ou des pertes d'opportunités commerciales.

Les approches classiques de prévision reposent souvent sur l'expérience des responsables, des moyennes historiques ou des extrapolations linéaires. Ces méthodes deviennent insuffisantes lorsque les données présentent une forte variabilité, une saisonnalité marquée, des comportements d'achat différents selon les segments clients et des effets liés aux décisions commerciales.

Problématique centrale

Comment concevoir une application web capable de transformer des données commerciales issues d'un contexte industriel B2B en prévisions de ventes exploitables, lisibles et orientées décision, tout en intégrant la segmentation client et la simulation de scénarios métier ?

2.2 Limites des approches existantes

Les outils classiques de reporting et de Business Intelligence permettent de visualiser les ventes passées, mais ils ne répondent pas toujours aux besoins d'anticipation. Ils décrivent ce qui s'est produit, sans toujours fournir une projection claire de ce qui peut arriver ni une aide concrète à la décision. Les solutions commerciales avancées existent, mais elles sont souvent coûteuses, complexes à intégrer et peu adaptées aux PME industrielles ou

aux contextes locaux.

Dans ce contexte, les principales limites concernent la dépendance forte à la qualité des données ERP, l'absence de segmentation B2B adaptée au secteur industriel, la difficulté d'expliquer les prévisions aux utilisateurs métier et le manque de modules de simulation accessibles. Ces limites justifient la conception d'une solution plus ciblée, plus lisible et mieux alignée avec les besoins des managers commerciaux.

2.3 Objectif général

L'objectif général de Previzma est de concevoir et développer une application web permettant de prévoir les ventes industrielles B2B à partir de données historiques enrichies par une couche métier, tout en fournissant une interface décisionnelle simple, claire et exploitable.

2.4 Objectifs spécifiques

Les objectifs spécifiques découlent directement de cette finalité. Le projet vise d'abord à modéliser les données commerciales nécessaires à la prévision et à intégrer une segmentation client B2B basée sur les comportements d'achat. Il vise ensuite à construire un pipeline de préparation des données adapté au Machine Learning, à entraîner un modèle de prévision basé sur XGBoost et à exposer ses prédictions via un microservice FastAPI.

Sur le plan applicatif, Previzma doit fournir un backend métier avec Spring Boot, un dashboard Angular lisible pour les managers, un module *What-If* de simulation d'impact et des indicateurs simples permettant de suivre les performances, les risques et les alertes opérationnelles.

2.5 Contributions attendues

Les contributions attendues du projet ne se limitent pas à la création d'un modèle prédictif. Elles incluent également la conception d'une architecture logicielle complète, la structuration d'une base de données métier, l'intégration d'une couche de simulation et la mise en place d'une interface de pilotage décisionnel. L'intérêt du projet réside dans la combinaison de ces éléments : transformer une donnée commerciale brute en information exploitable, puis restituer cette information sous une forme compréhensible par un décideur métier.

Chapitre 3

Étude théorique et état de l'art

3.1 Prédiction des ventes

La prédiction des ventes consiste à estimer les ventes futures d'une entreprise sur une période donnée. Elle peut être réalisée à différents niveaux : global, par produit, par catégorie, par région, par canal de distribution ou par segment client. Dans un contexte B2B, la prédiction doit tenir compte de cycles de vente plus longs, de volumes importants, de contrats récurrents et d'une forte dépendance aux grands clients.

Les approches de prédiction peuvent être regroupées en trois grandes familles. Les méthodes statistiques classiques, comme les moyennes mobiles, le lissage exponentiel ou ARIMA, sont utiles lorsque les séries sont relativement stables et régulières. Les méthodes de Machine Learning, comme Random Forest ou XGBoost, sont plus adaptées lorsque les ventes dépendent de plusieurs variables explicatives. Les méthodes de Deep Learning, comme les réseaux LSTM, peuvent traiter des dépendances temporelles complexes, mais elles demandent généralement plus de données et davantage de ressources de calcul.

3.2 Spécificités du B2B industriel

Le B2B industriel présente des caractéristiques différentes du commerce B2C ou du retail classique. Les clients sont moins nombreux mais génèrent souvent des volumes plus importants. Les achats peuvent dépendre de contrats, de calendriers de production, de contraintes logistiques ou de budgets périodiques.

Dans ce contexte, la segmentation client joue un rôle essentiel. Un grand compte peut commander de grandes quantités à intervalles réguliers, tandis qu'une PME peut acheter de manière plus ponctuelle. Un distributeur régional peut présenter une saisonnalité locale spécifique. Ces différences doivent être intégrées dans le modèle de prédiction afin d'éviter une estimation trop générale et peu exploitable.

3.3 Données ERP et données Machine Learning

Un ERP stocke des données opérationnelles : commandes, clients, produits, factures, statuts, stocks et mouvements comptables. Ces données sont structurées pour assurer le suivi de l'activité, la traçabilité et la gestion administrative. En revanche, un modèle de Machine Learning cherche des signaux prédictifs. La même donnée peut donc être pertinente pour la gestion courante, mais insuffisante sous sa forme brute pour la prévision.

Il est nécessaire de transformer les données ERP en variables exploitables. Les statuts de commande peuvent par exemple être convertis en variable indiquant si une commande est réellement confirmée. Les catégories tarifaires peuvent être remplacées par des segments comportementaux. Les prix et volumes peuvent être enrichis par des indicateurs de saisonnalité, de tendance ou d'historique d'achat.



Figure 3.1 – Passage des données ERP aux données exploitables par le Machine Learning

3.4 Algorithmes de prévision

3.4.1 ARIMA

ARIMA est un modèle statistique classique utilisé pour les séries temporelles stationnaires. Il combine une composante autorégressive, une différenciation et une moyenne mobile. Il est explicable et performant sur des séries régulières, mais il devient moins adapté lorsque les ventes dépendent de nombreuses variables métier ou présentent des relations non linéaires.

3.4.2 Lissage exponentiel

Le lissage exponentiel attribue un poids plus important aux observations récentes. Il est simple, rapide et utile pour des séries présentant une tendance relativement stable. Cependant, il devient limité lorsque les variations dépendent de facteurs externes comme le segment client, les conditions commerciales ou les effets de prix.

3.4.3 Random Forest

Random Forest est un algorithme d'ensemble basé sur plusieurs arbres de décision. Il offre une bonne robustesse et réduit le risque de surapprentissage. Il peut néanmoins devenir moins précis que des méthodes de boosting sur certains jeux de données tabulaires structurés.

3.4.4 XGBoost

XGBoost est un algorithme de gradient boosting basé sur des arbres de décision. Il est particulièrement adapté aux données tabulaires et permet de capturer des relations non linéaires entre les variables. Son efficacité, sa rapidité d'entraînement et sa compatibilité avec des méthodes d'interprétabilité comme SHAP en font un choix pertinent pour Previzma.

3.4.5 LSTM

Les réseaux LSTM sont des modèles de Deep Learning conçus pour traiter les dépendances temporelles longues. Ils peuvent être performants sur des séries complexes, mais nécessitent généralement plus de données, plus de ressources de calcul et une phase d'entraînement plus coûteuse. Pour un PFA orienté application web décisionnelle, ils représentent donc une option intéressante à comparer, mais moins prioritaire que XGBoost.

3.5 Comparaison synthétique des algorithmes

Table 3.1 – Comparaison synthétique des algorithmes de prévision

Algorithme	Forces	Faiblesses	Pertinence pour Previzma
ARIMA	Explicable, adapté aux séries stables	Peu adapté aux variables multiples	Moyenne
Lissage exponentiel	Simple et rapide	Limité sur données complexes	Faible à moyenne
Random Forest	Robuste, stable	Moins performant sur certains scénarios	Bonne
XGBoost	Précis, rapide, compatible SHAP	Nécessite un bon feature engineering	Très bonne
LSTM	Puissant sur dépendances longues	Coût élevé, besoin de données importantes	Moyenne

3.6 Solutions existantes

Le marché propose plusieurs solutions de prévision des ventes intégrées à des plateformes CRM, ERP ou Revenue Operations. Parmi les solutions connues figurent Salesforce Einstein, Microsoft Dynamics 365, Oracle Sales Cloud, Zoho CRM ou encore Clari. Ces outils offrent des fonctionnalités avancées, mais ils peuvent présenter des limites importantes pour les PME industrielles : coût élevé, complexité d'intégration, faible adaptation sectorielle et absence de simulation métier accessible.

3.7 Positionnement de Previzma

Previzma se positionne comme une solution académique orientée métier, conçue pour démontrer comment une architecture moderne peut transformer des données commerciales B2B en prévisions exploitables. Le projet se distingue par son orientation industrielle, sa segmentation client native, son module *What-If* et son effort de lisibilité pour les utilisateurs non techniques. Sa valeur ne réside donc pas uniquement dans le choix de l'algorithme, mais dans la chaîne complète allant de la donnée opérationnelle jusqu'à la décision commerciale.

Chapitre 4

Technologies utilisées

4.1 Vue d'ensemble de la stack technique

La stack technique de Previzma a été choisie selon quatre critères principaux : maintenabilité, scalabilité, séparation des responsabilités et adéquation avec les besoins du projet. Angular assure l'interface utilisateur, Spring Boot porte le backend métier, PostgreSQL/Supabase stocke les données structurées et FastAPI expose le moteur de prévision Python.

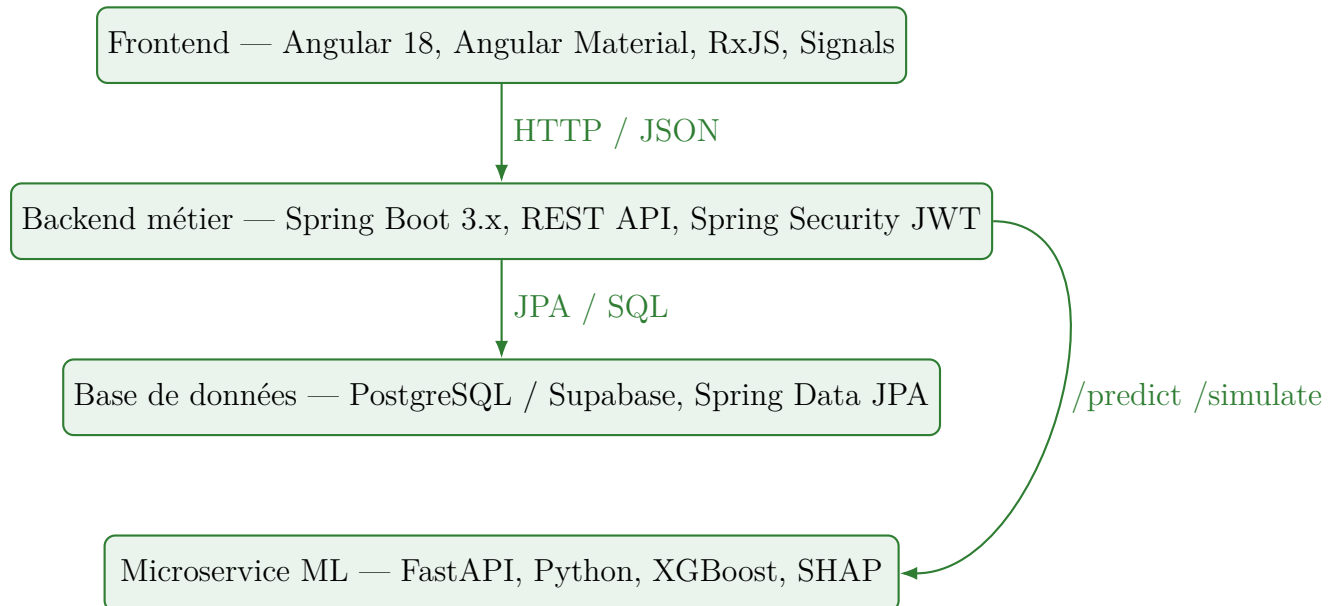


Figure 4.1 — Vue d'ensemble de la stack technique de Previzma

4.2 Angular 18

Angular est un framework frontend destiné au développement d'applications web modernes, modulaires et maintenables. Dans Previzma, il est utilisé pour construire le dashboard

décisionnel, les formulaires du module *What-If*, les vues de prévision, les filtres par segment et les composants de visualisation. Son intérêt réside dans sa structure stricte, ses composants réutilisables, ses formulaires réactifs et sa bonne adaptation aux applications orientées données.

4.3 Spring Boot 3.x

Spring Boot est utilisé comme backend métier principal. Il expose les API REST, gère l'authentification, applique les règles métier, orchestre les appels vers le microservice ML et assure l'accès à la base de données via Spring Data JPA. Ce choix permet de bénéficier d'un environnement Java robuste, adapté aux applications entreprise et aux architectures en couches.

4.4 PostgreSQL et Supabase

PostgreSQL est utilisé comme base de données relationnelle. Dans le cadre du projet, Supabase fournit une instance PostgreSQL cloud facilitant la mise en place, la consultation des tables et la validation rapide du modèle de données. La base permet de stocker les utilisateurs, les entreprises, les segments, les produits, les ventes historiques, les prévisions générées et les simulations exécutées.

4.5 FastAPI

FastAPI est utilisé pour exposer le moteur de prévision sous forme de microservice Python. Il permet de séparer la logique Machine Learning du backend métier Java. Cette séparation facilite l'évolution indépendante du modèle, les tests et le redéploiement du service de prévision sans impacter directement le reste de l'application.

4.6 XGBoost

XGBoost est retenu comme algorithme principal de prévision. Il est performant sur les données tabulaires, rapide à entraîner, capable de modéliser des relations non linéaires et compatible avec des techniques d'explicabilité telles que SHAP. Dans Previzma, il s'intègre naturellement au module *What-If*, car une simulation peut être représentée comme une modification contrôlée de certaines variables d'entrée.

4.7 Outils complémentaires

Le projet peut également s'appuyer sur Docker pour la conteneurisation, GitHub pour la gestion du code source, Postman pour les tests API, Supabase Studio pour l'administration de la base PostgreSQL et des bibliothèques Python comme pandas, NumPy, scikit-learn et SHAP pour le traitement des données.

Chapitre 5

Analyse et conception

5.1 Besoins fonctionnels

Les besoins fonctionnels de Previzma décrivent les principales fonctionnalités attendues par les utilisateurs. Ils couvrent l'authentification, la consultation du dashboard, la préparation des données, la génération des prévisions, le lancement des simulations, la conservation de l'historique et l'interprétation des résultats.

Table 5.1 – Besoins fonctionnels de Previzma

Code	Besoin fonctionnel
BF01	L'utilisateur doit pouvoir s'authentifier de manière sécurisée.
BF02	L'utilisateur doit pouvoir consulter un dashboard global des ventes et prévisions.
BF03	L'utilisateur doit pouvoir filtrer les résultats par période, produit et segment client.
BF04	Le système doit pouvoir générer des prévisions de ventes.
BF05	Le système doit permettre la segmentation B2B des clients.
BF06	Le système doit afficher des alertes opérationnelles selon des seuils configurés.
BF07	L'utilisateur doit pouvoir lancer une simulation What-If.
BF08	Le système doit conserver l'historique des prévisions et simulations.
BF09	L'utilisateur doit pouvoir comparer les ventes réelles avec les ventes prévues.

BF10	L'utilisateur doit pouvoir interpréter les résultats à travers des KPIs simples.
------	--

5.2 Besoins non fonctionnels

Les besoins non fonctionnels expriment les qualités attendues du système. Pour Previzma, les priorités sont la maintenabilité, la sécurité, la lisibilité pour l'utilisateur métier, l'évolutivité du microservice ML et la traçabilité des données historisées.

Table 5.2 – Besoins non fonctionnels de Previzma

Code	Besoin non fonctionnel
BNF01	L'application doit être modulaire et maintenable.
BNF02	Les API doivent être sécurisées par authentification JWT.
BNF03	Le dashboard doit être lisible par un utilisateur non technique.
BNF04	Le microservice ML doit pouvoir évoluer indépendamment du backend métier.
BNF05	Les données doivent être structurées et historisées.
BNF06	Le système doit permettre une réponse rapide pour les simulations What-If.
BNF07	L'architecture doit faciliter les tests et le déploiement.

5.3 Acteurs du système

Le système est utilisé par trois profils humains principaux. L'administrateur prend en charge la configuration de la plateforme, la gestion des utilisateurs, des produits, des segments et l'import des données. Le manager commercial consulte les indicateurs, les alertes, les prévisions et lance des simulations afin d'appuyer ses décisions. L'analyste métier approfondit l'interprétation des résultats, compare les scénarios et contribue à la compréhension des tendances commerciales.

Deux acteurs techniques complètent le système. La source de données, représentée par un dataset Kaggle enrichi ou par un ERP simulé, fournit les ventes historiques. Le microservice ML, construit avec FastAPI et XGBoost, calcule les prévisions et les simulations à partir des variables préparées par le backend.

5.4 Diagrammes de cas d'utilisation

Afin de préserver la lisibilité du modèle, les cas d'utilisation sont séparés en deux vues complémentaires. La première décrit les fonctionnalités d'administration et de gestion des données. La seconde décrit l'exploitation métier du système autour du dashboard, de la prévision et du module *What-If*.

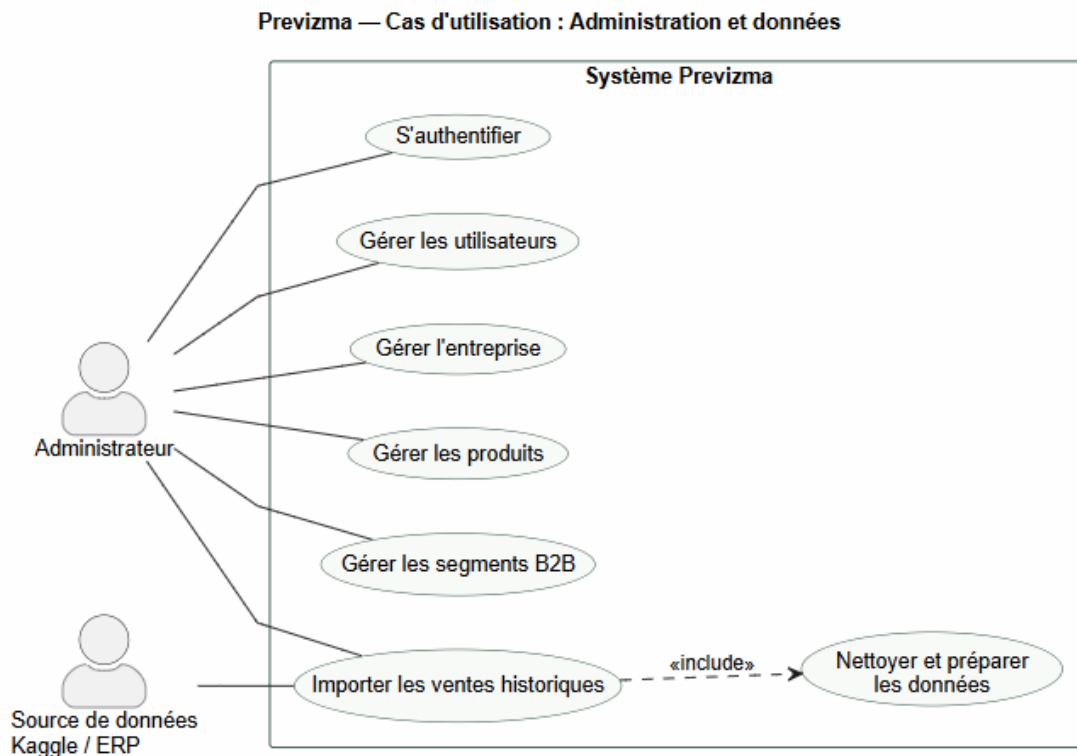


Figure 5.1 – Diagramme de cas d'utilisation – Administration et données

Le premier diagramme montre le rôle de l'administrateur dans la gestion de la plateforme. Il couvre l'authentification, la gestion des utilisateurs, la gestion de l'entreprise, la gestion des produits, la gestion des segments B2B ainsi que l'import et la préparation des ventes historiques.

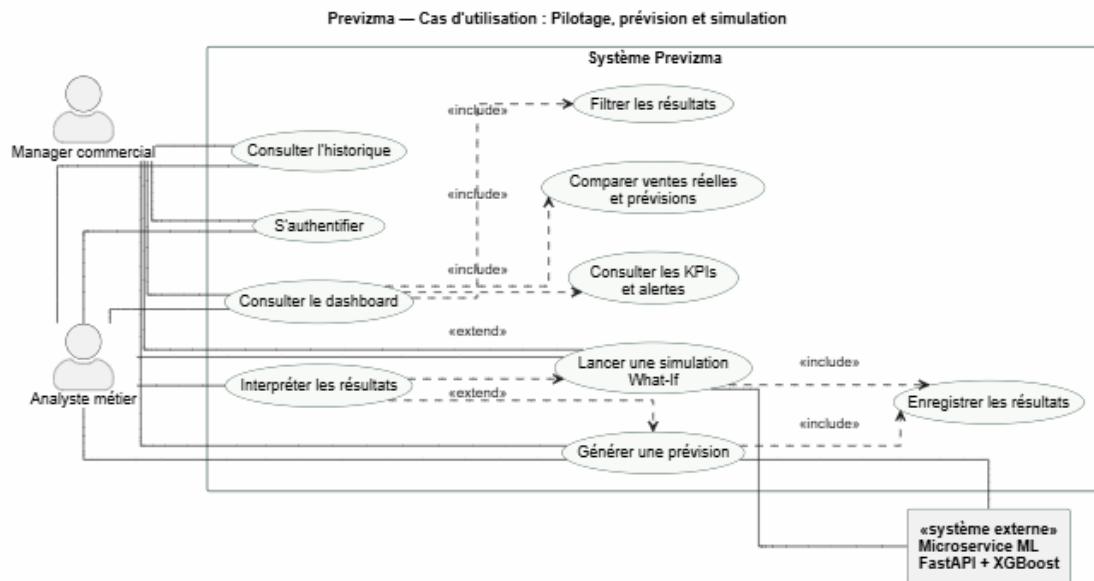


Figure 5.2 – Diagramme de cas d'utilisation – Pilotage, prévision et simulation

Le second diagramme présente les interactions principales du manager commercial et de l'analyste métier avec le système. Il met en évidence la consultation du dashboard, l'analyse des KPIs, la génération des prévisions, le lancement d'une simulation *What-If*, la consultation de l'historique et l'interprétation des résultats.

5.5 Diagramme de classes

Le diagramme de classes constitue la base de la conception orientée objet du backend Spring Boot. Il identifie les entités métier principales, leurs attributs et leurs associations. La classe **Company** représente l'organisation cliente et sert de racine fonctionnelle au modèle. Elle est associée aux utilisateurs, aux produits et aux segments clients. Les ventes historiques, les prévisions, les simulations et les alertes sont ensuite rattachées aux produits et aux segments afin de permettre une lecture décisionnelle par profil B2B.

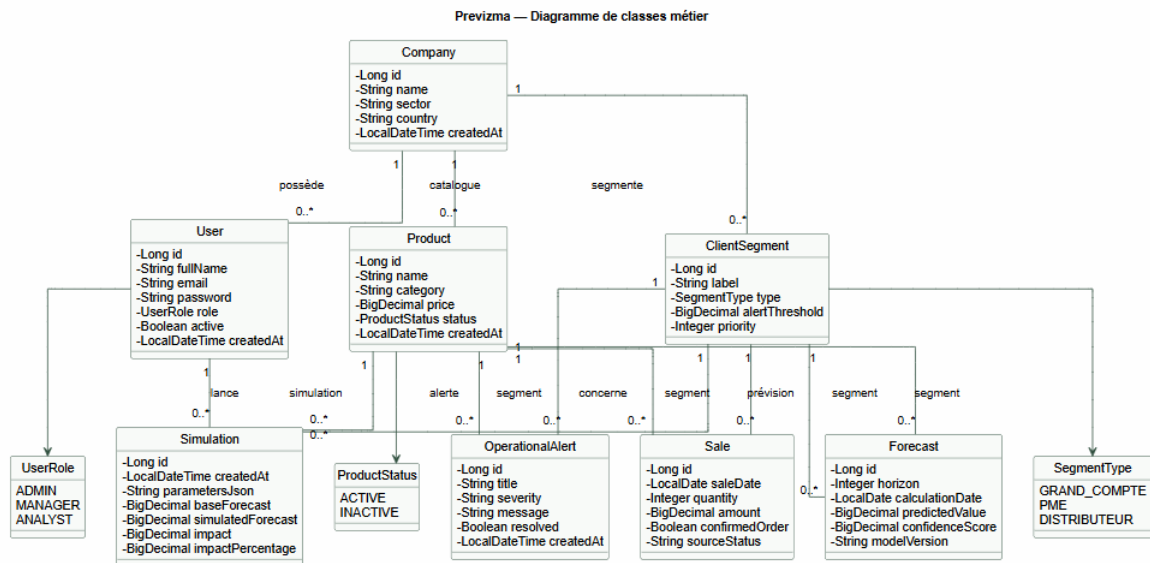


Figure 5.3 – Diagramme de classes métier de Previzma

Cette modélisation facilite la création des entités JPA dans Spring Boot. Les relations `Company-User`, `Company-Product` et `Company-ClientSegment` structurent le contexte métier. Les relations entre `Sale`, `Forecast`, `Simulation`, `Product` et `ClientSegment` permettent de conserver l'historique des ventes, les résultats de prévision et les scénarios simulés.

5.6 Modèle conceptuel de données

Le modèle conceptuel de données traduit le diagramme de classes en une structure relationnelle adaptée à PostgreSQL. Le choix d'un modèle relationnel est cohérent avec la nature des données manipulées : elles sont structurées, historisées et fortement liées entre elles. Les simulations utilisent une colonne JSON afin de stocker les paramètres du scénario sans complexifier excessivement le schéma principal.

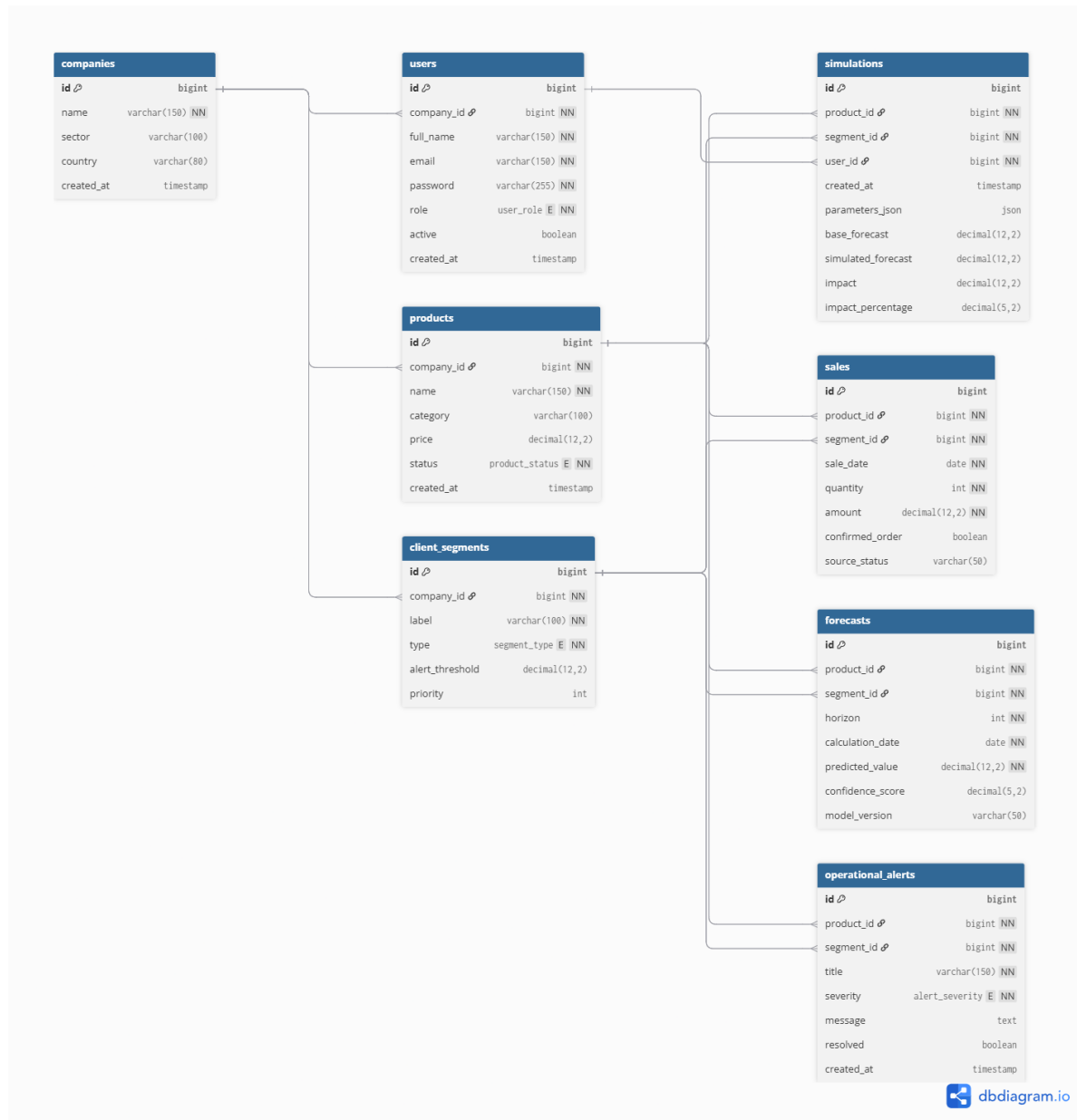


Figure 5.4 – Modèle conceptuel de données de Previzma

Table 5.3 – Entités principales de la base de données

Table	Rôle	Attributs principaux
companies	Entreprises clientes	id, name, sector, country, created_at
users	Gestion des comptes	id, company_id, full_name, email, password, role, active
products	Catalogue produits	id, company_id, name, category, price, status
client_segments	Segmentation B2B	id, company_id, label, type, alert_threshold, priority
sales	Historique des ventes	id, product_id, segment_id, sale_date, quantity, amount, confirmed_order
forecasts	Prévisions générées	id, product_id, segment_id, horizon, calculation_date, predicted_value
simulations	Scénarios What-If	id, product_id, segment_id, user_id, parameters_json, impact
operational_alerts	Alertes métier	id, product_id, segment_id, title, severity, message, resolved

5.7 Diagramme d’architecture globale

L’architecture globale de Previzma repose sur une séparation claire des responsabilités. Le frontend Angular assure l’interaction avec l’utilisateur, l’affichage du dashboard, les formulaires de simulation et la communication avec le backend via HTTP et JSON. Le backend Spring Boot centralise la sécurité, les règles métier, l’orchestration des prévisions et l’accès aux données. PostgreSQL/Supabase assure la persistance des entités métier. Le microservice FastAPI expose les endpoints de prédiction et de simulation en s’appuyant sur un modèle XGBoost.

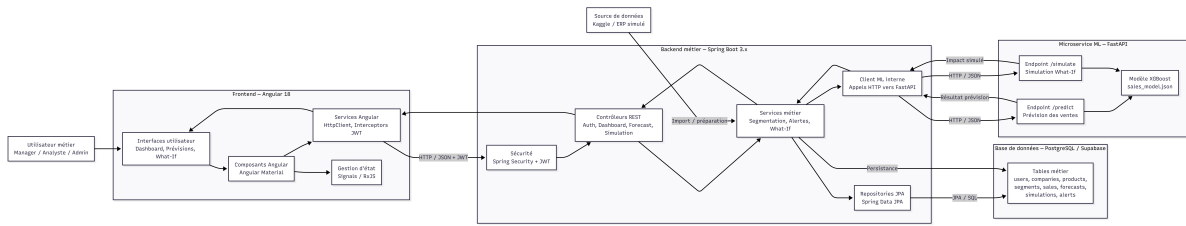


Figure 5.5 – Diagramme d'architecture globale de Previzma

5.8 Diagramme de séquence – Simulation What-If

Le diagramme de séquence présente le déroulement d'une simulation *What-If*. Le manager saisit les paramètres dans l'interface Angular. Le backend Spring Boot vérifie l'authentification, récupère les données métier nécessaires, construit les variables d'entrée puis interroge le microservice FastAPI. Le modèle XGBoost calcule la prévision de base et la prévision simulée. Le résultat est ensuite sauvegardé dans PostgreSQL et retourné au frontend.

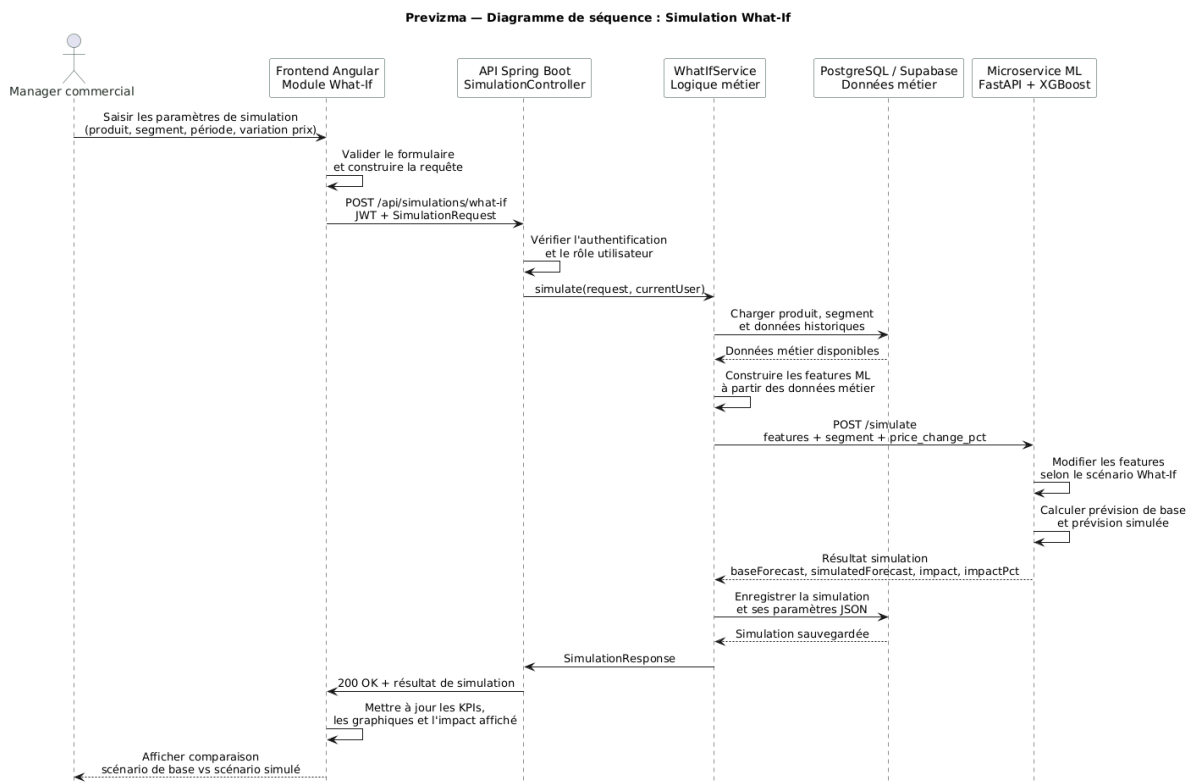


Figure 5.6 – Diagramme de séquence du module What-If

5.9 Pipeline de prévision

Le pipeline de prévision suit plusieurs étapes : collecte des ventes historiques, nettoyage des données, segmentation client, feature engineering, entraînement du modèle XGBoost,

génération des prévisions et restitution via l'interface. Cette chaîne de traitement montre que la performance ne dépend pas uniquement de l'algorithme, mais également de la qualité de transformation des données opérationnelles en variables exploitables.

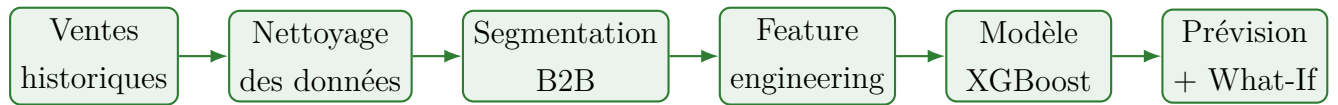


Figure 5.7 – Pipeline de prévision de Previzma

Chapitre 6

Réalisation de l'application

6.1 Préparation des données

La préparation des données constitue une étape essentielle du projet. Elle transforme les données brutes de ventes en variables exploitables par le modèle de prévision. Cette étape ne se limite pas à un nettoyage technique : elle traduit aussi la réalité métier B2B en signaux compréhensibles pour le modèle.

Concrètement, les statuts de commande sont normalisés afin de distinguer les commandes réellement confirmées des commandes annulées ou en brouillon. Les ventes sont agrégées par période, produit et segment client. Les variables temporelles, comme le mois, le trimestre ou la saison, sont ajoutées pour représenter les effets de calendrier. Les données de prix et de volume sont également préparées afin de permettre au module *What-If* de simuler l'impact d'une décision commerciale.

Placeholder d'exécution

Insérer ici une capture du notebook ou du script de préparation des données, avec un aperçu du dataset avant et après nettoyage.

6.2 Backend Spring Boot

Le backend Spring Boot constitue le cœur métier de l'application. Il expose les API REST consommées par Angular, gère l'authentification, applique les règles métier, communique avec PostgreSQL/Supabase et orchestre les appels au microservice ML. L'organisation retenue suit une architecture en couches : contrôleurs pour recevoir les requêtes, services pour appliquer la logique métier, repositories pour accéder aux données, et composants d'intégration pour appeler FastAPI.

Cette structuration est volontairement progressive afin de faciliter la prise en main de Spring Boot. La première étape consiste à construire les entités métier de base, puis à ajouter les services, les contrôleurs et les endpoints. La sécurité JWT est ensuite intégrée pour protéger les routes et différencier les accès selon les rôles.

Placeholder d'exécution

Insérer ici une capture d'IntelliJ montrant le démarrage du backend Spring Boot, la connexion à Supabase/PostgreSQL et les packages principaux du projet.

6.3 Microservice ML FastAPI

Le microservice ML est séparé du backend Java afin de conserver Python pour l'entraînement, le chargement et l'exécution du modèle XGBoost. Il expose deux services principaux : un service de prédiction standard et un service de simulation. Cette séparation permet de modifier ou réentraîner le modèle sans perturber directement le backend métier.

Le backend Spring Boot prépare les variables d'entrée, appelle le microservice FastAPI et récupère le résultat sous forme JSON. Cette réponse est ensuite transformée en objet métier, enregistrée si nécessaire et renvoyée au frontend.

Placeholder d'exécution

Insérer ici une capture du serveur FastAPI, de la documentation Swagger et d'un test de l'endpoint de prédiction.

6.4 Module What-If

Le module *What-If* permet de simuler l'impact d'une modification de paramètres métier sur les ventes prévues. Par exemple, un manager peut modifier le prix d'un produit ou ajuster un volume cible afin d'observer l'effet potentiel sur la demande prévue.

Le scénario de base et le scénario simulé sont comparés afin d'afficher un impact en valeur absolue et en pourcentage. Cette fonctionnalité est importante car elle transforme la prévision en outil d'aide à la décision. Le manager ne se contente pas de lire une valeur prédite : il peut explorer plusieurs décisions possibles et comprendre leur impact estimé.

Placeholder d'exécution

Insérer ici une capture du test de la simulation What-If avec un scénario de baisse de prix et l'impact retourné.

6.5 Frontend Angular

Le frontend Angular regroupe les composants nécessaires à l'expérience utilisateur : dashboard, filtres, graphiques, formulaires de simulation, cartes KPI et pages d'analyse. L'objectif principal du frontend est de réduire le temps de compréhension des résultats. L'utilisateur doit pouvoir identifier rapidement les tendances, les écarts et les alertes importantes.

L'interface est pensée pour un utilisateur métier et non pour un data scientist. Les informations doivent donc être synthétiques, hiérarchisées et visuellement claires. Les filtres par période, produit et segment permettent d'explorer les résultats sans surcharger l'écran.

Placeholder d'exécution

Insérer ici une capture de l'arborescence Angular, des composants principaux et du lancement de l'application avec `ng serve`.

6.6 Dashboard décisionnel

Le dashboard présente les indicateurs clés de pilotage : taux de réalisation, ventes prévues, ventes réelles, score de risque, alertes actives et précision du modèle. Ces informations sont affichées par segment client, produit et période afin de fournir une lecture opérationnelle exploitable.

L'intérêt du dashboard réside dans sa capacité à résumer une situation commerciale en quelques secondes. Un manager doit pouvoir identifier rapidement les produits à surveiller, les segments en croissance, les écarts entre réel et prévu et les alertes prioritaires.

Placeholder d'exécution

Insérer ici les captures d'écran du dashboard final : cartes KPI, courbe ventes réelles vs prévisions, filtres par période/produit/segment et alertes opérationnelles.

6.7 Sécurité et authentification

La sécurité est assurée au niveau du backend par Spring Security et JWT. L'utilisateur doit s'authentifier avant d'accéder aux fonctionnalités. Les rôles permettent de limiter l'accès à certaines opérations selon le profil utilisateur. L'administrateur peut gérer les référentiels et les utilisateurs, tandis que le manager et l'analyste accèdent principalement aux fonctionnalités de consultation, de prévision et de simulation.

Placeholder d'exécution

Insérer ici une capture de la page de connexion, de la réponse `/api/auth/login` et d'un appel API protégé avec token JWT.

Chapitre 7

Étude de cas et résultats

7.1 Présentation du dataset

Le projet s'appuie sur un dataset de ventes enrichi par une couche métier B2B. L'objectif n'est pas uniquement de prédire une série de ventes, mais de reproduire un contexte industriel dans lequel les clients appartiennent à des segments différents et où les décisions commerciales peuvent influencer les volumes futurs.

Placeholder d'exécution

À compléter : Ajouter le nom exact du dataset Kaggle, le nombre de lignes, les colonnes principales, la période couverte et les transformations B2B appliquées.

7.2 Scénario métier étudié

Le scénario de référence consiste à analyser les ventes d'un produit industriel sur plusieurs périodes, puis à générer une prévision par segment client. Les segments considérés sont les grands comptes, les PME et les distributeurs.

Un exemple de scénario consiste à prévoir les ventes mensuelles d'une gamme de produits industriels pour le trimestre suivant, puis à simuler l'impact d'une baisse de prix de 10% sur le segment des grands comptes. Ce scénario illustre la logique de Previzma : partir des données historiques, produire une prévision, puis transformer cette prévision en support de décision.

7.3 Indicateurs d'évaluation

Les performances du modèle sont évaluées à l'aide de métriques classiques de prévision. La MAE mesure l'erreur absolue moyenne, la RMSE pénalise davantage les erreurs importantes,

et la MAPE exprime l’erreur en pourcentage afin de faciliter l’interprétation métier. Ces métriques permettent de comparer plusieurs approches et de justifier le choix final du modèle.

Table 7.1 – Indicateurs d’évaluation du modèle

Métrique	Interprétation
MAE	Mesure l’erreur absolue moyenne entre les ventes réelles et les ventes prévues.
RMSE	Mesure l’erreur quadratique moyenne et pénalise davantage les erreurs importantes.
MAPE	Exprime l’erreur moyenne en pourcentage, ce qui facilite la lecture par les utilisateurs métier.

7.4 Résultats de prévision

Les résultats finaux seront intégrés après l’entraînement du modèle sur le dataset final. La comparaison devra présenter les performances par algorithme, mais aussi par segment client lorsque cela est possible. Cette lecture par segment est essentielle, car une bonne performance globale peut masquer des erreurs importantes sur un segment particulier.

Placeholder d’exécution

À compléter : Ajouter les résultats finaux du modèle : MAE, RMSE, MAPE, comparaison par segment et comparaison entre baseline et XGBoost.

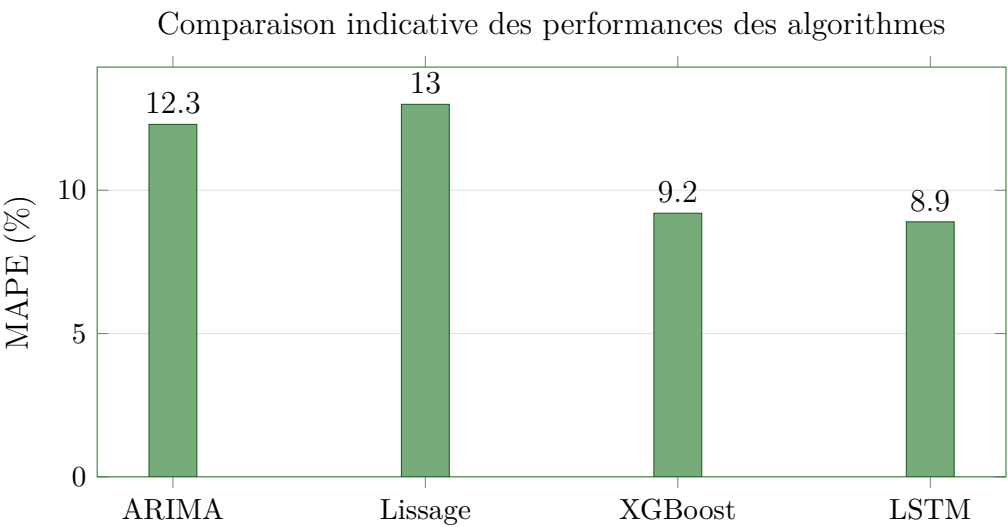


Figure 7.1 – Comparaison indicative des MAPE par algorithme

Placeholder d'exécution

Remplacer ou compléter ce graphique par les résultats obtenus après entraînement réel du modèle sur le dataset final.

7.5 Simulation What-If

Le module *What-If* permet de comparer un scénario de base avec un scénario simulé. Le résultat est présenté sous forme de variation absolue et de variation en pourcentage. Cette restitution aide l'utilisateur à comprendre l'effet estimé d'une décision commerciale avant de la mettre en œuvre.

Placeholder d'exécution

À compléter : Ajouter une capture d'écran du module What-If et un exemple chiffré final.

Placeholder d'exécution

Insérer ici un cas complet : produit sélectionné, segment client, prix initial, variation appliquée, prévision de base, prévision simulée, impact absolu et impact en pourcentage.

7.6 Interprétation des résultats

L'interprétation des résultats doit permettre de répondre à des questions métier : quels segments présentent la plus forte croissance prévue, quels produits risquent une rupture, et quelle décision commerciale a le plus d'impact sur les ventes prévues. Cette section sera rédigée après l'obtention des résultats finaux et l'ajout des captures du dashboard.

Placeholder d'exécution

À compléter : Rédiger cette section après obtention des résultats finaux et ajout des captures du dashboard.

Chapitre 8

Tests, limites et perspectives

8.1 Tests réalisés

Les tests prévus pour Previzma couvrent plusieurs niveaux : tests unitaires backend, tests d'intégration API, tests du microservice ML, tests de validation des données et tests fonctionnels de l'interface. Cette stratégie permet de vérifier à la fois la logique métier, la cohérence des échanges entre services et le comportement visible par l'utilisateur.

Table 8.1 – Plan de tests prévu

Type de test	Objectif	Outil possible
Tests unitaires backend	Vérifier les services métier	JUnit, Mockito
Tests API	Vérifier les endpoints REST	Postman, Swagger
Tests ML	Vérifier les prédictions	pytest, notebooks
Tests frontend	Vérifier les composants Angular	Angular Testing Tools
Tests fonctionnels	Vérifier les scénarios utilisateur	Tests manuels guidés

Placeholder d'exécution

Insérer ici les captures des tests Postman/Swagger, des tests unitaires backend et des tests du microservice ML lorsque l'application sera prête.

8.2 Limites du projet

Comme tout projet académique, Previzma présente certaines limites. Le dataset utilisé est enrichi pour reproduire un contexte B2B industriel, mais il ne remplace pas une validation complète sur des données réelles issues d'un ERP en production. Les résultats du modèle dépendent fortement de la qualité des données, du feature engineering et de la représentativité du dataset.

Une autre limite concerne la couverture fonctionnelle. Le projet vise à démontrer une architecture et un scénario métier cohérents dans un délai limité. Certaines fonctionnalités, comme l'intégration directe avec un ERP réel, la surveillance continue du modèle ou l'explicabilité détaillée via SHAP, peuvent être approfondies dans une version ultérieure.

8.3 Perspectives d'amélioration

Plusieurs améliorations peuvent être envisagées après la première version. Une connexion directe à un ERP tel qu'Odoo permettrait d'automatiser l'import des ventes et de réduire les traitements manuels. L'ajout d'une explicabilité plus avancée via SHAP renforcerait la confiance des utilisateurs métier. Une conteneurisation avec Docker faciliterait le déploiement de l'application complète, tandis qu'un module de recommandations commerciales pourrait transformer les prévisions en actions suggérées.

À plus long terme, Previzma pourrait évoluer vers une solution cloud démontrable en conditions réelles, avec suivi des performances du modèle, comparaison automatique de plusieurs algorithmes et amélioration continue des prévisions à partir des nouvelles données collectées.

Conclusion générale

Ce projet de fin d'année a permis de concevoir une application web de prévision des ventes adaptée au contexte de l'industrie B2B. À travers Previzma, nous avons cherché à répondre à une problématique réelle : transformer des données commerciales issues d'un environnement opérationnel en prévisions exploitables pour la prise de décision.

L'approche adoptée repose sur une architecture distribuée combinant Angular, Spring Boot, PostgreSQL/Supabase, FastAPI et XGBoost. Cette architecture permet de séparer les responsabilités entre l'interface utilisateur, le backend métier, la persistance des données et le moteur de prévision. Le projet met également en avant l'importance de la segmentation B2B, du feature engineering et de la simulation *What-If*.

Au-delà de l'aspect technique, Previzma illustre la nécessité de rapprocher les systèmes opérationnels des outils analytiques. Une prévision pertinente ne dépend pas uniquement du choix de l'algorithme ; elle dépend également de la qualité des données, de leur structuration et de leur traduction en variables métier significatives.

Les résultats attendus montrent que Previzma peut constituer une base solide pour développer une solution décisionnelle plus complète, capable d'aider les entreprises industrielles à anticiper leurs ventes, réduire les risques opérationnels et améliorer leur pilotage commercial.

Bibliographie

- [1] Deng et al., *Research on Enterprise Sales Forecast and Business Decision Support*, ACM, 2022.
- [2] Forma.ai, *Exactly How Enterprise Sales Forecasting Is Evolving for 2025*, Technical Report, 2024.
- [3] GetCompass, *7 Logiciels de prévision des ventes en 2025*, 2025.
- [4] Gupta and Agarwal, *Enhancing Sales Forecasting Accuracy through Integrated Machine Learning*, Semantic Scholar, 2024.
- [5] Singh et al., *From ARIMA to LSTM : Evaluating Traditional and AI-Based Models for Accurate Retail Sales Forecasting*, IJRASET, 2025.
- [6] Rashmi and Amardeep, *Machine Learning Based Predictive Market for Sales Forecasting*, IJSET, Vol. 12, 2024.
- [7] NVIDIA Developer Blog, *Building a Machine Learning Microservice with FastAPI*, 2022.
- [8] Vaadin, *The Most Popular Java Frameworks in 2026 : A Complete Guide*, 2026.
- [9] TestMu AI, *Angular vs React : A Complete Comparison for 2025*, 2025.
- [10] PostgreSQL Documentation, *PostgreSQL Official Documentation*.

Chapitre A

Annexes

A.1 Exemples d'endpoints API

Table A.1 – Endpoints principaux de l'application

Méthode	Endpoint	Description
POST	/api/auth/login	Authentification utilisateur
GET	/api/dashboard	Récupération des KPIs du dashboard
POST	/api/forecasts/predict	Génération d'une prévision
POST	/api/simulations/what-if	Exécution d'une simulation What-If
GET	/api/segments	Liste des segments B2B
GET	/api/alerts	Liste des alertes opérationnelles

A.2 Emplacements prévus pour les captures d'écran

Les captures finales devront documenter l'exécution réelle de l'application. Elles concerneront la page de connexion, le dashboard principal, la page des prévisions, le module *What-If*, les alertes opérationnelles, la documentation Swagger, l'arborescence Spring Boot, les tables Supabase/PostgreSQL, les tests Postman et les résultats finaux du modèle.

A.3 Noms recommandés pour les figures externes

Les fichiers suivants sont attendus dans le dossier `images/`. Il est recommandé d'éviter les espaces et les accents dans les noms de fichiers afin de réduire les risques d'erreurs de compilation LaTeX.

Table A.2 – Fichiers image attendus

Fichier	Contenu
emsih.png	Logo EMSI.
logo.png	Logo Previzma.
usecase-administration-donnees.png	Cas d'utilisation administration et données.
usecase-pilotage-prevision-whatif.png	Cas d'utilisation pilotage, prévision et simulation.
diagramme-classes-previzma.png	Diagramme de classes métier.
mcd-previzma.png	Modèle conceptuel de données.
architecture.png	Diagramme d'architecture globale.
sequence-whatif-previzma.png	Diagramme de séquence du module What-If.